

General Guidelines for Research Projects

1. **Originality:** The project must either propose a new optimization, a novel architectural integration, or a rigorous comparative performance analysis that does not exist in current literature.
2. **Implementation:** All projects require a working prototype (using C++, Java, or Python) and benchmarking against baseline systems.
3. **Paper Structure:** Final submissions must follow the **ACM SIGMOD** or **VLDB** conference templates, including: *Abstract, Introduction, Related Work, Methodology, Evaluation, and Conclusion*.

These projects are designed to bridge the gap between traditional database theory and cutting-edge AI-integrated systems, suitable for submission to conferences such as SIGMOD, VLDB, or ICDE.

Example Project Plans

Learned Indexing for Non-Uniform Workloads

- **Problem:** Traditional B-Trees and Hash indexes are data-agnostic. Learned indexes (using ML models) can predict data distribution to reduce memory footprint and lookup time, but they often struggle with high-frequency "write" operations (inserts/updates).
- **Directions for Solution:** Implement a "Hybrid Learned Index" that uses a small neural network or spline-based model for read-heavy segments of a dataset while maintaining a standard B-Tree "buffer" for new inserts.
- **Research Focus:** Investigate the trade-off between model retraining frequency and query latency.
- **Plan of Work:**
 - *Weeks 4-6:* Implement a basic Recursive Model Index (RMI).
 - *Weeks 7-9:* Integrate a "Write-Buffer" mechanism.
 - *Weeks 10-12:* Benchmark against standard PostgreSQL B-Trees and RocksDB LSM-trees.
 - *Weeks 13-14:* Paper writing and result visualization.

Self-Optimizing Partitioning for Distributed NoSQL

- **Problem:** Data skew in distributed systems (e.g., MongoDB or Cassandra) leads to "hot partitions" where one node is overloaded. Static sharding keys cannot adapt to changing query patterns.
- **Directions for Solution:** Develop a middleware that monitors query frequency per key range and uses a Reinforcement Learning (RL) agent to trigger live data migration/re-partitioning.

- **Research Focus:** Can RL-based partitioning outperform traditional consistent hashing in dynamic "Flash Crowd" scenarios?
- **Plan of Work:**
 - *Weeks 4-6:* Setup a 3-node distributed cluster.
 - *Weeks 7-9:* Develop the monitoring middleware and RL agent (using OpenAI Gym/PettingZoo).
 - *Weeks 10-12:* Simulate dynamic workloads and measure throughput stability.
 - *Weeks 13-14:* Statistical analysis for the paper.

Optimization of Vector Search in RAG Systems

- **Problem:** Retrieval-Augmented Generation (RAG) relies on Vector DBs. However, high-dimensional vector search (ANN) is computationally expensive and often returns irrelevant context if the embedding model and the DB retrieval strategy are not aligned.
- **Directions for Solution:** Propose a "Semantic-Aware Pruning" strategy for HNSW (Hierarchical Navigable Small World) graphs that uses LLM-generated metadata to filter the search space before vector calculation.
- **Research Focus:** Improving the "Recall@K" metric while reducing query latency in high-dimensional spaces.
- **Plan of Work:**
 - *Weeks 4-6:* Implement a standard RAG pipeline with Milvus or Pinecone.
 - *Weeks 7-9:* Design the metadata filtering layer.
 - *Weeks 10-12:* Evaluate accuracy vs. speed using the MTEB (Massive Text Embedding Benchmark) dataset.
 - *Weeks 13-14:* Final paper formatting.

Autonomous Index Selection for HTAP Workloads

- **Problem:** In Hybrid Transactional/Analytical Processing (HTAP), the system must handle both fast inserts and heavy scans. Manually selecting which columns to index is nearly impossible when workloads shift between OLTP and OLAP.
- **Directions for Solution:** Build an "Autonomous Advisor" that uses a "What-If" optimizer to simulate the impact of indexes on a shared memory pool without actually building them, then automatically deploys the most beneficial ones.
- **Research Focus:** Minimizing the "Index Selection Overhead" in real-time systems.
- **Plan of Work:**
 - *Weeks 4-6:* Modify an open-source HTAP engine (e.g., TiDB or DuckDB).
 - *Weeks 7-9:* Build the "What-If" cost estimator.
 - *Weeks 10-12:* Compare with the "Peloton" self-driving DB approach.
 - *Weeks 13-14:* Documentation of findings.

Consistency vs. Latency in Geo-Distributed Parallel DBs

- **Problem:** In globally distributed databases, maintaining Strong Consistency (Strict Serializability) across continents leads to massive latency.
- **Directions for Solution:** Implement a "Tiered Consistency" model where the database automatically switches between Eventual and Strong consistency based on the transaction type (e.g., "Add to Cart" vs. "Final Payment").
- **Research Focus:** Quantifying the "Cost of Consistency" in terms of dollar-value/user-retention vs. network overhead.
- **Plan of Work:**
 - *Weeks 4-6:* Deploy a database across multiple AWS/Azure regions.
 - *Weeks 7-9:* Implement the consistency-switching logic in the transaction manager.
 - *Weeks 10-12:* Run YCSB (Yahoo! Cloud Serving Benchmark) across regions.
 - *Weeks 13-14:* Finalize the research manuscript.

Learned Index Structures for Dynamic High-Write Workloads

- **The Problem:** Current "Learned Indexes" (like RMI) outperform B-Trees in read-heavy scenarios by approximating data distributions. However, they suffer from high "retraining costs" when data is frequently inserted or updated, making them impractical for real-time transactional systems.
- **Directions for Solution and Research:** * Research "updatable" learned indexes that utilize a delta-buffer or a linearized structure.
 - Proposed Solution: Implement a two-tier architecture where new writes are handled by a traditional LSM-tree component, while the base data is managed by a learned model. Research the "optimal merge frequency" to maintain read speed without stalling writes.
- **Plan of Work:**
 - **Phase 1 (Weeks 4-6):** Implement a baseline B-Tree and a standard Recursive Model Index (RMI).
 - **Phase 2 (Weeks 7-9):** Develop the "Delta-Buffer" mechanism and the background model retraining logic.
 - **Phase 3 (Weeks 10-12):** Benchmark against TPC-C (write-heavy) and TPC-H (read-heavy) workloads.
 - **Phase 4 (Weeks 13-14):** Write the manuscript focusing on the latency trade-offs.

Self-Optimizing Partitioning in Distributed NoSQL Systems

- **The Problem:** Distributed databases often suffer from "data skew," where a few nodes handle the majority of requests (Hot Spots). Static partitioning (Hashing/Ranging) fails when access patterns change dynamically.
- **Directions for Solution and Research:**

- Research Reinforcement Learning (RL) agents for database administration.
- Proposed Solution: Build a middleware layer that monitors partition "heat." Use a Deep Q-Network (DQN) to predict future hot spots and trigger proactive data migration between shards before the node hits capacity.
- **Plan of Work:**
 - **Phase 1 (Weeks 4-6):** Setup a multi-node cluster (using MongoDB or Cassandra) and generate a skewed workload.
 - **Phase 2 (Weeks 7-9):** Design the monitoring agent and the RL reward function (based on throughput and CPU utilization).
 - **Phase 3 (Weeks 10-12):** Compare RL-based partitioning against Consistent Hashing.
 - **Phase 4 (Weeks 13-14):** Analyze the "cost of migration" versus "performance gain" for the journal paper.

Efficient Hybrid Vector-Relational Search for RAG Systems

- **The Problem:** Modern Retrieval-Augmented Generation (RAG) applications require both vector search (for semantic similarity) and relational filtering (e.g., "Find documents similar to X *but only from the year 2023*"). Performing these as two separate steps (Pre-filtering or Post-filtering) is often inefficient.
- **Directions for Solution and Research:**
 - Research "Single-stage" hybrid indexing structures.
 - Proposed Solution: Implement an index that merges HNSW (Hierarchical Navigable Small World) graphs with filtered bitmasks or specialized relational metadata nodes within the graph itself.
- **Plan of Work:**
 - **Phase 1 (Weeks 4-6):** Implement a standard RAG pipeline using a vector store (Milvus/Faiss) and a relational DB (PostgreSQL).
 - **Phase 2 (Weeks 7-9):** Develop the unified index prototype that supports "constrained" vector search.
 - **Phase 3 (Weeks 10-12):** Evaluate Recall and Query Latency using the MTEB (Massive Text Embedding Benchmark).
 - **Phase 4 (Weeks 13-14):** Format results into the ACM conference style.

Autonomous Index Advisor for HTAP Workloads

- **The Problem:** Hybrid Transactional/Analytical Processing (HTAP) workloads require different indexes for the "T" (Transactional) and "A" (Analytical) parts. Maintaining all indexes consumes too much storage and slows down writes.
- **Directions for Solution and Research:**
 - Research "What-if" analysis in query optimizers.

- Proposed Solution: Create an autonomous advisor that uses a cost-model to drop and create indexes dynamically based on the current "Workload Shift." It must balance the "Benefit of an index for a scan" vs. "The overhead of updating that index during a write."
- **Plan of Work:**
 - **Phase 1 (Weeks 4-6):** Build a workload classifier that distinguishes between OLTP and OLAP queries in real-time.
 - **Phase 2 (Weeks 7-9):** Implement the "Index Utility" scoring algorithm.
 - **Phase 3 (Weeks 10-12):** Run the system against the CH-benCHmark (a standard HTAP benchmark).
 - **Phase 4 (Weeks 13-14):** Finalize the paper, focusing on the reduction in storage footprint.

Consistency-Aware Query Routing in Geo-Distributed Databases

- **The Problem:** In globally distributed systems, enforcing "Strong Consistency" (Linearizability) across continents causes high latency. However, many applications only need strong consistency for *some* transactions (e.g., payment) and can tolerate "Eventual Consistency" for others (e.g., product reviews).
- **Directions for Solution and Research:**
 - Research Adaptive Consistency models and TLA+ specifications for distributed protocols.
 - Proposed Solution: Develop a "Consistency-Aware Router" that analyzes incoming SQL queries and automatically routes them to the nearest "Local-Consistency" node or the "Global-Consensus" leader based on the required safety level.
- **Plan of Work:**
 - **Phase 1 (Weeks 4-6):** Deploy a distributed database (e.g., CockroachDB or YugabyteDB) across at least three simulated geographic regions.
 - **Phase 2 (Weeks 7-9):** Implement the query classifier and the routing logic.
 - **Phase 3 (Weeks 10-12):** Measure the "tail latency" (p99) reduction for mixed-consistency workloads.
 - **Phase 4 (Weeks 13-14):** Document the protocol and results for submission.

Alternative Application Projects

1. **Optimizing Semantic Search: A Comparative Study of Vector Indexing in Postgres vs. Pinecone:**
Explores how traditional relational systems with vector extensions (pgvector) compare to native vector databases regarding latency and recall for LLM embeddings.
2. **Autonomous Query Tuning: Implementing Reinforcement Learning for Automatic Index Selection:** A project focused on building a "self-driving" database component that predicts and creates optimal indexes based on changing analytical workloads.

3. **HTAP Performance Isolation: Evaluating Transactional Contention in DuckDB vs. TiDB:** Investigates how hybrid transactional/analytical processing systems maintain low latency for small writes while running massive aggregation queries simultaneously.
4. **Learned Index Structures for Multi-Dimensional Geospatial Data:** Extends the concept of "Learned Indexes" (using neural networks as replacements for B-Trees) to handle complex spatial data more efficiently than standard R-Trees.
5. **Distributed Consistency Trade-offs: A Validation of PACELC Theory in Cloud-Native NoSQL:** An experimental evaluation of modern NoSQL systems (like Cassandra or DynamoDB) to measure the exact latency cost of choosing "Strong Consistency" over "Eventual Consistency."
6. **Declarative AI Analytics: Building Semantic Operators for Automated Data Cleaning:** Implements a system where users can write SQL-like queries that use LLMs to automatically resolve entity ambiguity or fill missing values during ingestion.
7. **Graph-Relational Hybrid Architectures for Real-Time Fraud Detection:** Develops a system that replicates transactional data into a graph database (e.g., Neo4j) to perform sub-second multi-hop relationship analysis for fraud identification.
8. **Serverless Database Scalability: Quantifying the Impact of Cold-Starts on Query Latency:** Analyzes how modern serverless DBMS (like Neon or Amazon Aurora Serverless) handle rapid scaling events and the resulting performance penalties.
9. **Blockchain as a Verifiable Database: Evaluating Query Performance in BigChainDB:** Compares the throughput and query capabilities of blockchain-based data management systems against traditional distributed relational databases.
10. **Natural Language to SQL: Evaluating Accuracy Guarantees for Complex Relational Joins:** A research project focused on creating a benchmark for LLM-based SQL generation, specifically focusing on the "accuracy guarantees" required for enterprise-grade database reporting.

Potential Project Titles

11. **Optimizing Vector Similarity Search:** An evaluation of Graph-based vs. Quantization-based indexing for LLM memory.
12. **Autonomous Query Tuning:** A Reinforcement Learning approach to automated index selection in Cloud DBMS.
13. **HTAP Engine Benchmarking:** Analyzing the impact of analytical OLAP contention on real-time OLTP performance.
14. **Learned Index Structures for NoSQL:** Implementing B-Tree replacements using Neural Networks in Key-Value stores.
15. **Distributed Consistency in the Wild:** An empirical study of CAP/PACELC trade-offs in globally distributed databases.
16. **Declarative AI Analytics:** Implementing "Lotus-style" semantic operators for automated data cleaning with LLMs.

17. **Graph Database Scalability:** A comparative implementation of PageRank on Neo4j vs. specialized parallel graph engines.
18. **Privacy-Preserving SQL:** Evaluating the performance overhead of Differential Privacy in modern relational databases.
19. **Serverless Database Cold-Starts:** Architectural optimizations for minimizing latency in multi-tenant cloud environments.
20. **Blockchain Data Management:** Designing a high-throughput storage layer for decentralized ledger systems.